

Many clinics, one patient

What MedPin becomes when many practices use it. **A patient who has never installed anything finds every doctor they have seen waiting for them when they log in** — and no practice can read a record it was not given.

The shape

Rahul, 54 — diabetes and a heart problem.

Priya at Dr. Dey's desk types his phone number. Amit at Dr. Sen's desk, across town, types the same number. Neither knows about the other.

Three weeks later Rahul installs the app, enters his number, and both doctors are there. He never registered anywhere.

That works because identity, relationship and record are three different things with three different owners.

LAYER	OWNER	HOLDS
Login	The person. One phone number, globally.	Name, phone, language, devices
Patient	A body. One login may hold several.	Age, height, conditions, allergies
Enrollment	A patient at a practice.	Since when, under which doctor, active or revoked
Record	The practice, via the enrollment.	Prescriptions, notes, chat, alerts

Already true

User.phone is unique: true , so one person is already one account however many practices type their number. PatientProfile already exists as a separate table. The join is most of the way built.

models/User.js:31 · models/PatientProfile.js:10

The mechanism: the desk does not create a patient, it finds or creates a login by phone number and adds an enrollment. A second practice typing the same number adds a second enrollment, never a second account.

Every technique, and why

TECHNIQUE	WHAT IT SOLVES
Phone number as the identity	A patient finds all their doctors without ever registering. The number was the match all along, so there is no matching step to get wrong.
Find-or-create, never fail	A second practice typing the same number adds a link instead of erroring. One person stays one account.
SMS OTP for consent	Proves the number is answered by the person at the desk. A typo simply never produces a code — instead of silently attaching a practice to a stranger's record.
Enrollment as a join table	Makes "which practice does this belong to?" a required field. Forgetting the scope crashes instead of leaking.
Practice separate from location	Two branches of one practice share one logo, one registration and one set of patients. Two practices at one address stay separate tenants.
Patient separate from login	One phone carries a mother, a child and a grandmother — and the child can leave with their history at eighteen.
Membership as a row	A doctor can be head of one practice and a member of another. A leaver is an ended row, not a deleted account.
Conditions instead of columns	Adding asthma stops being a schema change. Home cards and triage rules attach to the illness, so the app fits the patient.
Delegated creation, gated prescribing	You verify once per practice instead of once per receptionist — but the one act with legal weight still waits for you.
Modular monolith	One developer, and one prescription touches five subsystems. Splitting that across services buys nothing and risks a patient with medicines and no reminders.

Who creates what

Delegated. You verify the practice; the head doctor builds the team.

CREATED	BY	GATE
Practice	MedPin	Registration verified before ACTIVE
Location	Head doctor	None — it is their own practice's address
Doctor	Head doctor	Works immediately; cannot prescribe until MedPin verifies the registration number
Front desk, dietician	Head doctor	Phone OTP — the colleague's own handset answers
Patient	Front desk	Phone OTP — the patient's own handset answers
Family member	Patient, or front desk	None — added under the login that cares for them

Practice, not location. The head doctor may open a second branch of the practice you verified. They may not create a new practice — that is a new entity needing verification.

One table doing two jobs

Clinic currently carries the practice — tagline , registrationNo , logos — and a building — addressLine , weeklyHours , slotMinutes . That is why *Salt Lake* and *Behala Evening* sat as two clinics, each with its own copy of one logo and one registration number.

models/Clinic.js:51-109

CASE	PRACTICES	LOCATIONS	DOCTORS
Dr. Dey today	1	2	1
Polyclinic, one building, eight departments	1	1	8
Chain, three branches	1	3	20
Two practices sharing an address	2	2	—

Membership is a row, not a rank

A clinician's link to a practice carries **role** (head or member) and **canAdmin** . Dr. Sen is head at his own practice and a member at the polyclinic; a practice manager gets **canAdmin** without prescribing. A doctor leaving is an ended membership — their prescriptions stay, their access stops.

Two things this needs

Two heads minimum before a practice activates. One head who leaves, retires or dies is a practice that can never add a receptionist again.

The platform role lives elsewhere. Not a flag on a user in this app — a separate admin page, own login, own audit log, no clinical data. A bug in the clinic app must not reach every practice you have.

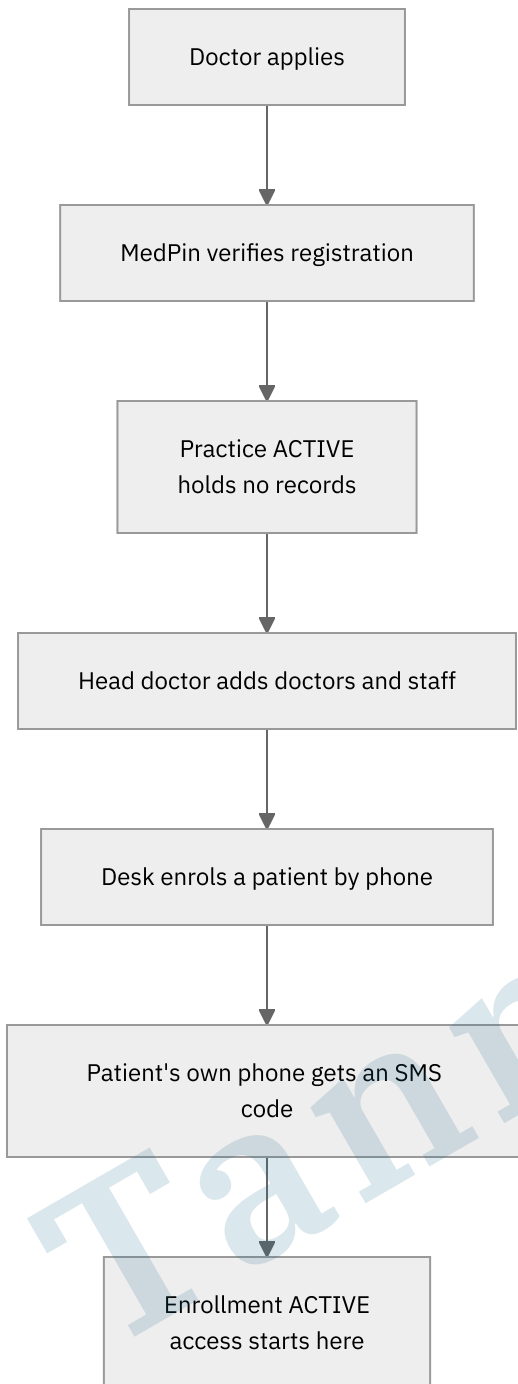
Onboarding, three phases

1. **By hand.** You create the practice and head doctor. Correct while small.
2. **Applications.** A doctor submits details; it lands `PENDING` ; you approve or reject. The typing moves, the judgement stays.
3. **Automated,** only if a registry can confirm a registration number. Check whether India's National Medical Register exposes something callable *before* planning around it — if not, phase 2 is a good end state.

The two gates

Verification is not access

An `ACTIVE` practice has been checked. It has not been given anybody's records. A fully verified practice still cannot type a phone number and read a stranger's history.



You hold the first gate. The patient holds the second. Passing the first grants nothing.

PENDING ACTIVE SUSPENDED REJECTED

PENDING ACTIVE ENDED

PENDING ACTIVE REVOKED

REVOKED deletes nothing. It ends future access. The prescriptions written remain — a record one party can erase is not a record.

The diagram illustrates the following classes and their attributes:

- PLATFORM_ADMIN**: (Abstract class)
- USER**: string phone, UK, string role, bool hasLoggedIn
- PRACTICE**: string name, objectid headDoctor, string status, string regNumber
- MEMBERSHIP**: objectid clinician, objectid practice, string role, preset name, json permissions, the actual grant, string status
- DEPARTMENT**: objectid practice, string name, json homeCards, json triageRules
- PATIENT**: objectid login, string name, date dateOfBirth, string relationship
- SHARE_GRANT**: objectid patient, objectid fromPractice, objectid toPractice, string scope, date expiresAt
- PATIENT_CONDITION**: objectid patient, objectid condition, string status, json detail
- CONDITION**: string key, json names, json homeCards, json triageRules
- LOCATION**: objectid practice, string addressLine, string mapUrl
- AVAILABILITY**: objectid doctor, objectid location, json weeklyHours, int slotMinutes
- ENROLLMENT**: objectid patient, objectid practice, string status, date enrolledOn
- DOCTOR_DEPARTMENT**: objectid doctor, objectid department
- THREAD**: (Abstract class)
- CONSENT_EVENT**: objectid enrollment, string action, granted revoked, objectid actor, string method, ota_desk_in_app, date at
- PRESCRIPTION**: (Abstract class)
- APPOINTMENT**: (Abstract class)
- RECORD**: (Abstract class)

Key relationships include:

- PLATFORM_ADMIN** verifies **PRACTICE**.
- USER** works in **MEMBERSHIP** and is staffed by **PRACTICE**.
- PRACTICE** defines **DEPARTMENT** and cares for **PATIENT**.
- MEMBERSHIP** is staffed by **DEPARTMENT**.
- DEPARTMENT** grants **SHARE_GRANT**.
- PATIENT** has **PATIENT_CONDITION**.
- CONDITION** classifies **PATIENT_CONDITION** and keeps **AVAILABILITY**.
- LOCATION** hosts **AVAILABILITY** and links **ENROLLMENT**.
- AVAILABILITY** is linked by **ENROLLMENT**.
- ENROLLMENT** has **PRESCRIPTION**, **APPOINTMENT**, and **RECORD**.
- ENROLLMENT** is logged by **CONSENT_EVENT**.
- THREAD** scopes **SHARE_GRANT**.

6 / 16

Indexes

COLLECTION	INDEX	SERVES
users	phone unique	Login, and the find-or-create join
patients	login	The family switcher
patient_conditions	patient + status	Which Home cards, which triage rules
enrollments	patient + clinic unique	No double enrollment
enrollments	patient + status	The patient's chat list — most-run query
memberships	clinician + practice unique	"May they add staff here?" — read on every request
doctor_departments	department	"Who can answer cardiology?"
consent_events	enrollment + at	The consent history for one relationship
audit_logs	subjectPatient + at	"Who has looked at this patient?"
availability	doctor + location unique	Slot generation
appointments	enrollment + scheduledFor	Today's list, day-before reminder
records	enrollment + kind + takenAt	Every graph on Home

Conditions replace the diabetes column

`diabetesType` stops being a field on the patient and becomes detail on a `PATIENT_CONDITION` row — Type 1 versus Type 2 is a fact about the diabetes, not about Rahul. `Condition` is the master list; each one declares its Home cards and its triage rules.

Migration cost

`diabetesType` is read in **48 places** — 12 API, 36 app — and `comorbidities` already exists as loose strings. Write to both, move readers a screen at a time, delete the column only when a search proves nothing reads it.

`models/PatientProfile.js:23, 57`

Structure versus JSON: if you would ever put it in a `where` clause, it is a column. JSON only for condition-specific detail nobody queries.

Rules that must hold

Six invariants. Each has a test attached before the feature it guards.

1. **Every clinical query names an enrollment.** Make it required at the schema level so a prescription without one cannot save. Forgetting the scope becomes a crash, not a leak.
2. **Access is not retroactive.** A practice reads from `enrolledOn` forward. Linking today does not open four years of another doctor's notes; the patient shares those deliberately or not at all.
3. **Never branch on count in the data layer.** Always fetch the list — of practices, patients, conditions. It may have one item. Only the screen counts.
4. **Every reminder names its person.** "Aarav · Syrup 5ml", never "Time for your medicine", on a phone carrying three people's prescriptions.
5. **The assistant sees one enrollment.** A question asked in the cardiologist's thread is answered from the cardiologist's record, and the patient's age and conditions go in the prompt — a four-year-old is not a small adult.
6. **Clinic-authored knowledge is filtered before ranking, not after.** A passage ranked then discarded has already been read.
7. **Nothing clinical is hard-deleted.** Voided, corrected or superseded, each with who and why. A record one party can erase is not a record.
8. **Every route is audited.** Reads included — a test fails if a new clinical route ships without it. Knowing who *looked* is the half usually missing.

The test that is the product

Two practices, one shared patient, and a test that fails if either can see the other's records. Write it before the second practice exists. Nothing ships to a real second customer until it is green.

Permissions and audit

Nine changes from review. Four were already in the code, better than the document described them.

Departments are rows, and a doctor may hold several

Not a hardcoded list of eight. An admin creates a department; each one carries its own Home cards and triage rules, so adding Neurology is data, not a deploy.

And the link is many-to-many: a physician covering General Medicine and Diabetology is one doctor in two departments, not two records.

One limit worth keeping

An admin may create a department freely, but its **triage rules start empty** and stay empty until a clinician writes them. A new department with inherited red flags is worse than one with none — see the rule about honest triage.

Permissions on the membership, presets by role

`role`, `canAdmin` and `canPrescribe` do not extend. Membership carries a permission set instead:

```
VIEW_PATIENT · EDIT_RECORD · PRESCRIBE · MANAGE_STAFF · MANAGE_DEPARTMENT ·  
VIEW_AUDIT · SHARE_RECORDS
```

Model it now, do not build the editor

Store the set from day one — retrofitting it later is a migration across every route. But seed it from three presets (head, clinician, desk) and ship **no permission-editing screen** until a customer asks.

A permission matrix nobody has asked for is a support burden and a way to lock a clinic out of its own records on a Sunday.

Authorisation is one function, server-side

Every clinical request answers five questions before it touches data:

1. Who is the user?
2. Which practice is this request about?
3. Is their membership there `ACTIVE` ?
4. Is the patient's enrollment there `ACTIVE` , and is the record after `enrolledOn` ?
5. Does their permission set allow this action?

One middleware, not five checks scattered per route. The app hiding a button is a courtesy; this is the control.

Consent is a log, not a field

An enrollment's `status` says where things stand now. It cannot answer "who granted this, when, and on what basis" — which is the question asked when something goes wrong.

So consent events append: `grantedAt` , `grantedBy` , `revokedAt` , `revokedBy` , method (OTP at desk, patient in app), and the wording version they agreed to. The enrollment's status becomes a projection of the latest event.

Sharing between practices is its own object

Deferred as a decision, but the shape is fixed so nothing has to be unpicked later: a **share grant** names the patient, the practice sharing, the practice receiving, what is shared, and when it expires. Granted by the patient, revocable by the patient, never implied by an enrollment.

Threads: department, with participants

The document said one thread per department and stopped there, which breaks when a department has four doctors.

Not one thread per doctor

A thread belongs to (**enrollment, department**). Any clinician in that department may answer, and every message carries its author. A patient messaging cardiology and getting whoever is on duty is better served than one waiting on a named doctor who is on leave.

The patient sees "Cardiology — Dr. Sen replied". The continuity they need is the department's; the accountability is the author's.

Audit — mostly already built

Better than the document said

AuditLog already records actor, actor role, action, resource, resource id, **subject patient**, IP, user agent and timestamp. Reads are already logged on **31** routes and writes on **63**.

What is missing is not the logging. It is (a) coverage — every new route must be wrapped, which a test can enforce — and (b) somewhere to read it. An audit trail nobody can query is a legal artefact, not a safety one.

models/AuditLog.js:10-22

Add: a practice-scoped audit view behind `VIEW_AUDIT`, and consent and sharing changes as logged actions.

Clinical records are never hard-deleted

Three states instead of a delete: **VOIDED** (issued in error), **CORRECTED** (replaced, both kept), **SUPERSEDED** (a later version exists). Each carries who, when and why.

The pattern is already here

`Prescription.supersedes` exists, with a comment saying a change writes a new version pointing at the original rather than mutating it. `MediaAsset` soft-deletes with `deletedAt`. This generalises what the codebase already does in two places.

models/Prescription.js:5, 78 · models/MediaAsset.js:58

Break-glass — designed, not built

An unconscious patient, a doctor who is not theirs. Real, and rare enough that building it now would be speculation. What is fixed now is the shape, so it can be added without redesign: **a reason is mandatory, access expires automatically, every read is logged, and the patient is told it happened.**

That last one is what makes it safe. Access that notifies the person accessed is access nobody abuses twice.

How the panels behave

Patient

PRACTICES	HOME	CHAT
None	Setup card: what should we track? Readings and medicines once added.	Nothing. No assistant by default.
One	Exactly today's screen	Opens straight into the thread
Several	Same cards, labelled by practice	A list, like WhatsApp

Cards come from conditions. Diabetes brings the sugar chart and HbA1c tile; hypertension brings blood pressure; asthma brings peak flow. No condition, no cards — which is the answer to empty sections.

Nothing appears empty. No Medicines list until there is a medicine, no Documents until there is a document, no Alarms until something needs one. Each appears when its first row exists and never leaves.

Threads split by department, not doctor. "The heart doctor's chat" and "the sugar doctor's chat". One department means one thread — today's case.

Family

A switcher appears only when a login holds more than one patient. Priya at the desk answers one extra question: is this for them, or someone they look after? Renu never touches a phone; her record lives under the login of whoever brings her.

Build detach at the same time. Aarav turns eighteen: his profile becomes its own login, keeping every reading and enrollment. The practices see no change, because their enrollments point at the profile.

Clinic

Unchanged in shape. What changes: the desk's roll and the doctor's day are scoped to the practice; a doctor without a verified registration sees a banner instead of a prescribe button; adding staff is a head-doctor or `canAdmin` action.

Architecture and scale

Modular monolith. Not microservices: one developer, and your operations are single transactions — sending a prescription writes it, creates medications, computes times, arms alarms and pushes. Across services that is a distributed transaction whose failure mode is a patient with medicines and no reminders.

Extract exactly two things, neither a microservice. The scheduler, for correctness — crons run inside the API, so two workers means every reminder fires twice. Later the AI worker, so a Gemini outage cannot take the appointment book down.

Order of scaling

1. Scheduler into its own single-instance process. **First**, or the next step doubles every reminder.
2. `pm2 -i max` — you are on one core of the machine you already pay for.
3. Uploads to S3-compatible storage. This is what actually blocks a second app server.
4. MongoDB onto its own VPS, off the box shared with sixteen apps.
5. Nginx in front of two app servers. No code change — the app is already stateless.

Vector search: in-process cosine is right to roughly 50,000 chunks; both backends are already written. Filter by practice before comparing. Move to pgvector or Qdrant on the same VPS when the corpus, not the traffic, demands it.

Add Sentry. You currently learn about bugs by being told. With ten practices you will not be.

Refuse: Kubernetes, microservices, a queue, GraphQL. Each answers a problem you do not have and costs the speed you do need.

Still open

Three. The rest are decided and written above.

A · Sharing between practices

Can Dr. Sen see Dr. Dey's prescriptions for Rahul? Default no. If yes, the patient grants it per practice and can withdraw it — never the practice that asks.

B · Pricing

Per practice, per doctor seat, or per patient? Cheapest correct start: flat per practice, no metering. Metering is its own project.

C · Dr. Dey's clinic

Migrate into the platform, or leave running while it is built beside? Recommendation: **leave it**. Real patients depend on it. Merge after a second practice has run end to end in staging.

Decided: enrollment is consented by SMS OTP; triage rules attach to a condition; the model is delegated with a prescribing gate; departments are rows a doctor may hold several of.

Tanmoy Das

The twenty recommendations, checked

#	RECOMMENDATION	STATUS
1	Dynamic departments	Adopted — was a hardcoded list of eight
2	Doctor ↔ department many-to-many	Adopted — was one field on the doctor
3	Granular permissions	Adopted, with the editor deferred
4	Backend authorisation	Was a rule; now a five-question middleware
5	Consent history	Adopted — was a status field only
6	Practice-to-practice sharing	Shape fixed; the decision stays open
7	Thread redesign	Adopted — department plus participants
8	Audit logging	Already built. Gap is coverage and a viewer
9	No hard deletion	Adopted — generalises supersedes
10	Multiple admins	Already specified — two heads minimum
11	Practice vs location	Already specified — its own section
12	Break-glass access	Adopted as shape, deferred as build
13	Modular monolith	Already specified
14	No early queue	Already specified
15	No Kubernetes	Already specified
16	Scheduler duplicate protection	Already specified — step 1 of scaling
17	AI worker separation	Already specified
18	Object storage	Already specified — step 3
19	Horizontal scaling	Already specified — step 5
20	Phased infrastructure	Already specified — the ordered list

Nine changed the design. Eight were already written. Three were already true in the code and described too modestly.

Build order

Each step leaves the app working. The first five are invisible to patients.

1 Departments

A table, seeded with the eight specialties in three languages, plus the doctor-to-department join. Useful with one doctor, and adding a ninth is data.

2 Practice split from location

Salt Lake and Behala become one practice, two locations, one logo, one registration number.

3 Admin surface

Separate page, own login: create practice, verify registration, activate. No clinical data.

4 Memberships and permissions

A row per clinician per practice, carrying a permission set seeded from three presets. Dr. Dey gets one marked head. No editing screen yet.

5 Authorisation middleware

The five questions in one place, applied to every clinical route. Before any of the multi-practice work, so nothing is written against the old assumption.

6 Conditions

Three tables written alongside `diabetesType`, readers moved a screen at a time.

7 Hours to the doctor

Availability per doctor per location; the slot engine reads it. Backfill one row from Dr. Dey's existing hours.

8 Patients separate from logins

Drop the unique constraint; enrollments point at the profile. Families work; detach becomes possible.

9 Enrollment

Every clinical query routed through it. One practice means identical behaviour — and the isolation test can now be written.

10 Remove the six singleton doctor lookups

After this the code stops believing there is one doctor.

11 Find-or-create by phone, with OTP consent

The desk stops failing on an existing number. The join starts working.

12 Unified chat list

Grouped by enrollment. One practice renders as today — the proof it is right.

13 Second practice in staging

Two practices, one shared patient, isolation test green. Only then a real second customer.

14 Scaling

In the order given above.

Written against commit `d002d06`. Every file and line reference was read in the source: the six singleton doctor lookups, the two retrieval backends, the 48 readers of `diabetesType`, and the one table carrying both a practice and a building.